

INDUSTRY INTERNSHIP
SUMMARY REPORT
GENC-NEXT INTERNSHIP
AT COGNIZANT

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING
Specialization in Artificial Intelligence

Submitted by
AVI JAIN (22SCSE1410010)

Submitted to
Ms. Jyoti
(Assistant Professor)



SCHOOL OF COMPUTING SCIENCE AND ENGINEERING
GREATER NOIDA, UTTAR PRADESH
SUMMER 2025-2026



03-Feb-2026

Candidate ID: 40263761

Avi Jain
B.Tech Computer Science & Engineering
Galgotias University

Dear Avi Jain,

Further to our Letter of Intent for the position of Programmer Analyst Trainee / Programmer Analyst aligned to the hiring category, we are pleased to offer you an internship with us at Cognizant office for **a period of 3 to 6 months**. Your internship on-boarding will be scheduled based on your availability, factoring your college exam schedule, availability of your Provisional Certificate and our business requirements.

During this period, you will be provided with a stipend of **INR 15,000/-** equated to the planned duration of the Internship curriculum and will be paid monthly based on your attendance, subject to eligibility for the monthly payroll processing for a given month, except for the last month's stipend, which will be paid upon successful completion of the learning curriculum.

Though Cognizant Internship is a skilling program aimed at enhancing technical acumen, it does not guarantee employment and there is no employer – employee relationship during the course of this internship program. However, the successful completion of internship may form a critical part of your eligibility for employment with Cognizant if an opportunity arises in future.

You will be provided a learning curriculum as per the skill track assigned to you. The learning design would expect you to drive your learning through hands on exercise and project work. There will also be a series of webinars, quizzes, Subject Matter Expertise (SME) interactions, mentor connects, code challenges, assessments etc. to accelerate your learning. The outcome during Internship would be monitored through formal evaluations.

Prior to joining on the rolls of Cognizant, if offered, you must have successfully completed the prescribed Internship program. In the event of unsatisfactory performance during the Internship or non-completion of the Internship, no Internship Completion Certificate shall be issued by Cognizant. Cognizant reserves rights at its sole discretion to revoke its Letter of Intent.

TABLE OF CONTENTS		
CHAPTER	TITLE	PAGE NO
	Abstract	
	List of Figures & List of Tables	
	List of Abbreviations	
1	Introduction	
	1.1 Objective of the Internship	
	1.2 Problem Statement and Research Objectives	
	1.3 Description of Domain	
	1.4 A Brief Introduction about the Organization	
2	Technical Description	
3	System Design	
	3.1 General Architecture	
	3.2 Design Phase	
	3.2.1 Data Flow Diagram	
	3.2.2 UML Diagrams	
	3.3 Methodology	
4	System Implementation	
5	Results and Discussions	
6	Conclusion and Future Work	
7	Appendices	
	7.1 Source Code	
	7.2 Learning Experiences	
	7.3 SWOT Analysis	

ABSTRACT

This report presents the work carried out during an industry internship undertaken at Cognizant Technology Solutions in the Quality Engineering & Assurance – Software Development Engineer in Test (QEA-SDET) domain, in the role of Programmer-Analyst Intern. The internship spanned from 17 February 2026 to 12 June 2026 and was structured around modern software-engineering practices, with a strong emphasis on building production-grade applications and validating them through automated testing.

The central deliverable of the internship was the design and development of the Skill-Based Task Allocation System, an application that automatically assigns work items to employees based on the match between a task's required skills and an employee's recorded skill set. The system was built using a full-stack technology suite comprising Java and Spring Boot on the server side, Angular on the client side, and a relational SQL database for persistence. Quality was enforced end-to-end through Selenium and Cucumber for behaviour-driven automated testing, while GitHub and Jenkins supported version control and continuous integration.

The project was executed using the Agile methodology, organised into iterative two-week sprints with regular planning, review, and retrospective ceremonies. Beyond the technical artefacts, the internship offered substantial exposure to professional engineering discipline, including code review culture, test-first development, and collaborative teamwork. The candidate cleared the Qualifier assessment with a score of 83 out of 100, achieved a Green status in the interim Red-Amber-Green (RAG) evaluation, and secured a Green status in the final evaluation on the first attempt. This report documents the objectives, technical foundations, system design, implementation details, results, and reflective learning outcomes of the internship.

LIST OF FIGURES

S. No	Fig. No	Title	Page No
1	3.1	Layered System Architecture	
2	3.2	Data Flow Diagram (Level 1)	
3	3.3	UML Use Case Diagram	
4	3.4	UML Class Diagram	
5	3.5	Agile Sprint Workflow	

LIST OF TABLES

S. No	Table No	Title	Page No
1	2.1	Technology Stack Summary	
2	4.1	Core Modules Implemented	
3	5.1	Evaluation Outcomes	

LIST OF ABBREVIATIONS

QEA	Quality Engineering & Assurance
SDET	Software Development Engineer in Test
API	Application Programming Interface
REST	Representational State Transfer
SPA	Single Page Application
JPA	Java Persistence API
SQL	Structured Query Language
UML	Unified Modeling Language
DFD	Data Flow Diagram
CI/CD	Continuous Integration / Continuous Deployment
BDD	Behaviour-Driven Development
RAG	Red, Amber, Green (Status Rating)
UI	User Interface
CRUD	Create, Read, Update, Delete

CHAPTER 1

INTRODUCTION

Modern technology organisations handle a continuous flow of work items — development tasks, testing assignments, bug fixes, and feature requests — that must be distributed across teams of people with varying skills. When this distribution is done manually, it is slow, prone to human bias, and often results in a poor fit between the demands of a task and the abilities of the person assigned to it. The internship documented in this report was centred on solving exactly this problem through software, while simultaneously building professional competence in full-stack development and automated quality engineering.

1.1 Objective of the Internship

The primary objective of the internship was to gain hands-on, industry-grade experience in the complete software development life cycle within the QEA-SDET domain, and to apply that experience to the construction of a real, working application. The specific objectives were as follows:

1. To understand and practise the end-to-end software development life cycle followed in a professional engineering environment.
2. To develop full-stack proficiency by building both the server side (Java, Spring Boot) and the client side (Angular) of an application.
3. To master automated testing practices using Selenium and Cucumber, in keeping with the SDET focus of the role.
4. To work within an Agile team, participating in sprint planning, daily stand-ups, reviews, and retrospectives.
5. To use industry tooling for version control (GitHub) and continuous integration (Jenkins) as part of everyday development.
6. To design, implement, test, and deliver a complete project — the Skill-Based Task Allocation System — meeting the quality bar set by the organisation's evaluation framework.

1.2 Problem Statement and Research Objectives

In a typical project team, a manager must decide which employee should handle each incoming task. The natural approach is to match the skills a task demands against the skills each employee possesses. Done by hand, this matching is error-prone: managers may overlook a well-suited team member, overload their most visible performers, or assign work to someone lacking the required expertise. The result is delayed delivery, uneven workload, and reduced quality.

Problem Statement: There is a need for a software system that can store the skill profiles of employees and the skill requirements of tasks, and then automatically recommend or assign the most suitable employee for each task, thereby removing manual guesswork and improving the accuracy and fairness of task distribution.

The research objectives that guided the project were to determine how employee skills and task requirements could be modelled in a relational database; how a matching rule could be expressed in business logic to rank candidates for a task; and how the whole workflow could be delivered as a responsive web application that both managers and employees could use comfortably.

1.3 Description of Domain

The internship was situated in the QEA-SDET domain. Quality Engineering & Assurance (QEA) is the discipline concerned with ensuring that software behaves correctly, reliably, and as specified before it reaches users. A Software Development Engineer in Test (SDET) is a hybrid role: an engineer who can both write production application code and design the automated tests that verify it. Unlike a traditional manual tester, an SDET treats test automation as a software-engineering problem in its own right, writing reusable test frameworks and integrating them into the delivery pipeline.

This domain demands a dual skill set. On one side sits application development — building features with languages and frameworks such as Java, Spring Boot, and Angular. On the other sits test automation — using tools such as Selenium for browser-level interaction and Cucumber for behaviour-driven specifications written in plain, readable language. The SDET works across both, ensuring that every feature delivered is accompanied by automated checks that guard against regressions, which are faults that reappear after previously working code is changed.

1.4 A Brief Introduction about the Organization

Brief History: Cognizant Technology Solutions is a multinational information-technology services and consulting company. Founded in 1994, it began as an in-house technology unit and subsequently grew into one of the world's large, publicly listed IT services organisations, serving clients across multiple industries worldwide.

Business Size: As a major publicly listed enterprise, Cognizant employs a very large global workforce numbering in the hundreds of thousands of professionals, distributed across delivery centres in many countries. This scale allows it to serve large enterprise clients across multiple time zones.

Product Lines and Services: The organisation offers a broad portfolio of services that includes digital engineering and software development, quality engineering and assurance, cloud and infrastructure services, data and analytics, artificial-intelligence solutions, business-process services, and technology consulting. Its work spans industries such as banking and financial services, healthcare, insurance, retail, manufacturing, and communications.

Competitors: Cognizant operates in a highly competitive global IT services market. Its principal competitors include other large technology and consulting firms such as Tata Consultancy Services, Infosys, Wipro, Accenture, and Capgemini, among others.

Summary of Departments: The organisation is structured into functions that work in concert: engineering and delivery teams that build and maintain client software; quality-engineering teams (the function in which this internship took place) that validate it; consulting and client-relationship teams that define solutions; and supporting functions such as human resources, learning and development, and operations that enable the delivery teams. During the internship, the candidate was embedded within an engineering and quality-engineering delivery team and collaborated closely with senior engineers and mentors.

CHAPTER 2

TECHNICAL DESCRIPTION

This chapter describes the technologies that formed the foundation of the internship work and the Skill-Based Task Allocation System. Each technology was selected because it represents an industry standard within full-stack development and automated quality engineering, and together they cover the complete path from the user interface down to the database and across to the testing and delivery pipeline.

Java: Java served as the primary programming language for the server side of the application. It is a strongly typed, object-oriented, platform-independent language widely used for building large and maintainable enterprise systems. Its rich ecosystem and stability made it the natural choice for the application's core logic.

Spring Boot: Spring Boot is a Java framework that simplifies the creation of stand-alone, production-ready applications. It was used to build the REST API layer of the system, providing controllers, services, and dependency injection with minimal configuration. Spring Data JPA, part of the wider Spring family, handled object-relational mapping between Java objects and database tables.

Angular: Angular is a TypeScript-based front-end framework used to build the Single Page Application (SPA) that forms the user interface. It allowed the construction of dynamic, component-based screens through which managers create tasks and allocate them, and through which employees view and update their assignments.

SQL: A relational SQL database provided persistent storage for the system. Structured Query Language was used to define the schema and to query and manipulate data relating to employees, skills, tasks, and allocations.

Selenium: Selenium is a browser-automation tool used to write tests that interact with the web application exactly as a real user would — clicking buttons, filling forms, and verifying on-screen results. It was central to validating the Angular front end automatically.

Cucumber: Cucumber is a Behaviour-Driven Development (BDD) tool that allows test scenarios to be written in a plain, structured English format known as Gherkin. These human-readable

scenarios were linked to automation code, making the tests understandable to both technical and non-technical stakeholders.

GitHub: GitHub provided distributed version control and collaboration. All source code was committed, branched, and reviewed through pull requests, supporting safe parallel work and a clear history of changes.

Jenkins: Jenkins is a continuous-integration server. It was configured to automatically build the application and run the automated test suite whenever new code was integrated, giving fast feedback on the health of the codebase.

Table 2.1 — Technology Stack Summary

Layer / Concern	Technology	Purpose
Front End	Angular (TypeScript)	Interactive single-page user interface
Back End	Java + Spring Boot	REST API, business logic, services
Persistence	SQL + Spring Data JPA	Storing employees, skills, tasks
Test Automation	Selenium + Cucumber	BDD end-to-end UI testing
Version Control	GitHub	Source control and code review
CI / Build	Jenkins	Automated build and test runs

CHAPTER 3

SYSTEM DESIGN

This chapter presents the design of the Skill-Based Task Allocation System. It describes the overall architecture of the application, the design artefacts produced during the planning phase — including the data flow diagram and the relevant UML diagrams — and the methodology under which the system was built.

3.1 General Architecture

The system follows a classic layered architecture. This separation of concerns means that each layer has a single, well-defined responsibility and communicates only with the layers directly adjacent to it, which makes the application easier to understand, test, and maintain. The Presentation Layer, built in Angular, runs in the user’s browser and handles all interaction. It communicates over HTTP with the Application Layer, a set of Spring Boot REST controllers and services that expose the system’s operations. Within this sits the Business Logic Layer, which houses the skill-matching and task-allocation engine — the heart of the system. Beneath it, the Data Access Layer uses Spring Data JPA repositories to read from and write to the relational SQL database. This structure is illustrated in Figure 3.1.

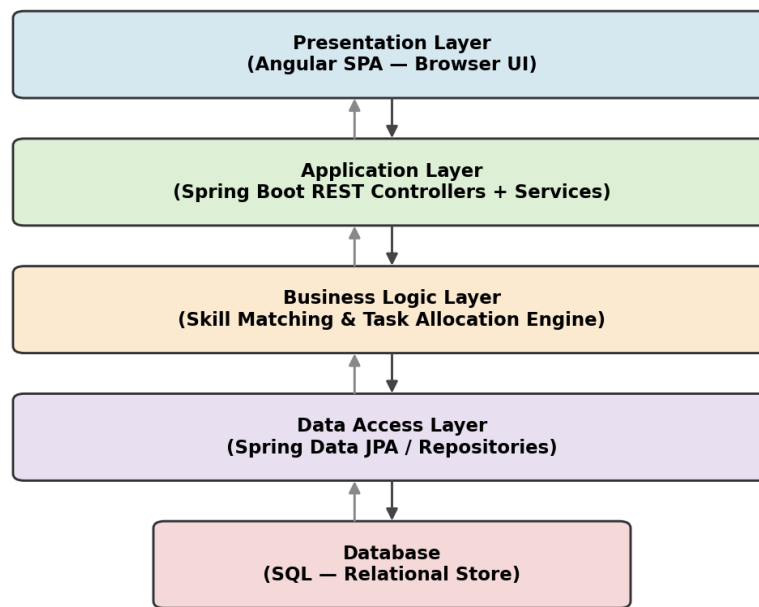


Figure 3.1 — Layered System Architecture

3.2 Design Phase

During the design phase, the team produced models of how data would move through the system and how the system's components would relate to one another. These models guided the implementation and served as a shared reference for the whole team.

3.2.1 Data Flow Diagram

The Data Flow Diagram (DFD) shows how information moves between the external users of the system, the processes that transform it, and the data stores that hold it. As shown in Figure 3.2, the Manager supplies task and skill data, which is stored and then used by the allocation process to match a task against available skills. The matched assignment flows to the Employee, who can update its status, with each allocation recorded in the data store.

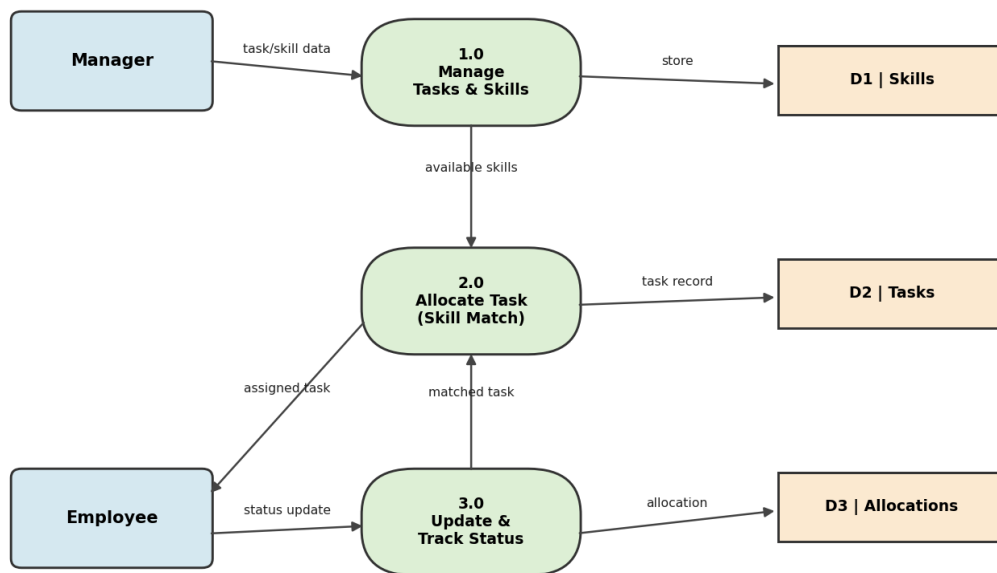


Figure 3.2 — Data Flow Diagram (Level 1)

3.2.2 UML Diagrams

Two Unified Modeling Language (UML) diagrams were prepared. The Use Case Diagram in Figure 3.3 captures what each kind of user can do with the system. The Manager can add or edit skills, create tasks, allocate tasks by skill, and view reports; the Employee can view assigned tasks and update their status.

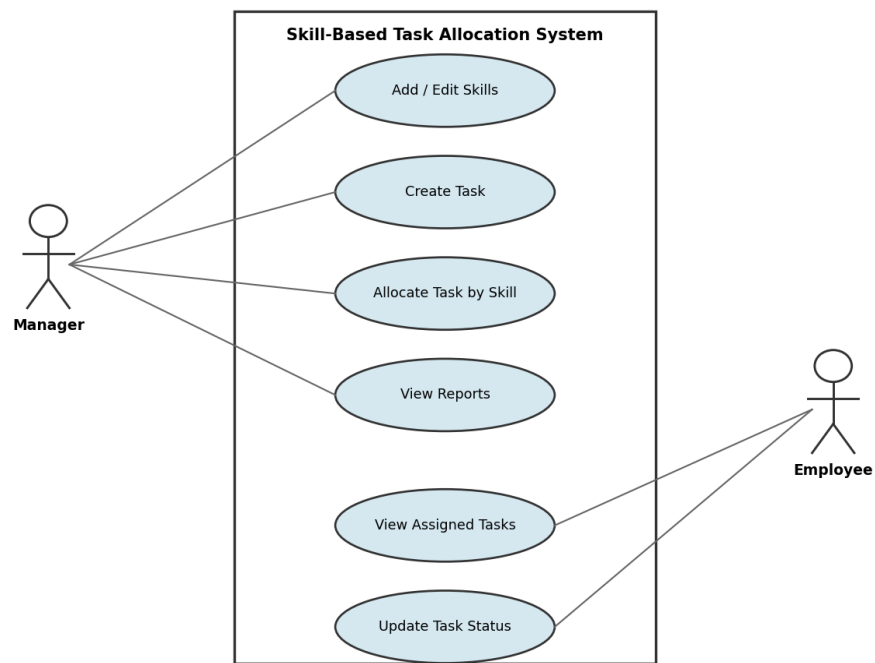


Figure 3.3 — UML Use Case Diagram

The Class Diagram in Figure 3.4 describes the main objects in the system and how they relate. An Employee holds a list of Skills; a Task requires a particular Skill and carries a status; an Allocation links a Task to the Employee assigned to it; and the AllocationService contains the logic that matches skills and performs the allocation.

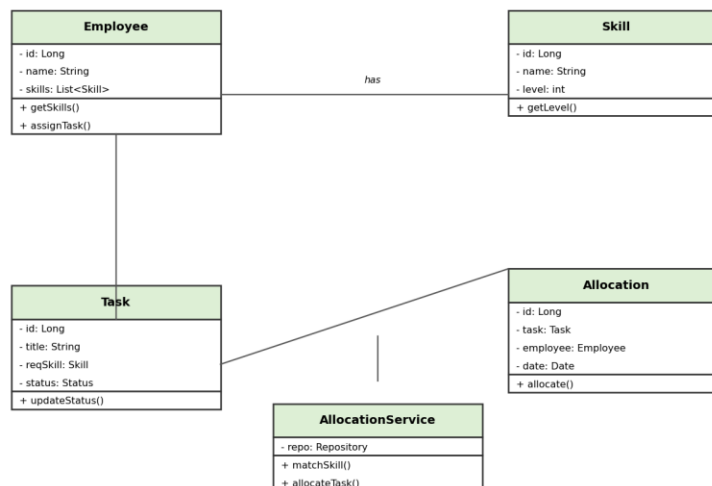
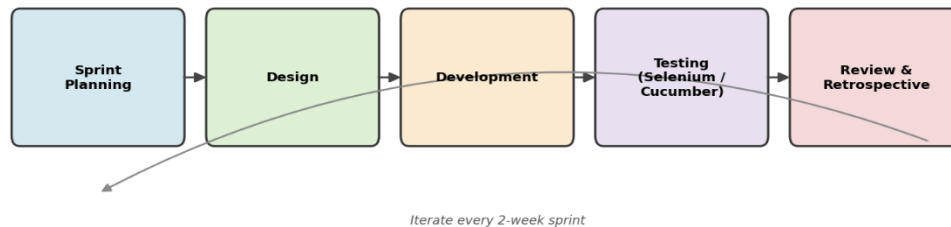


Figure 3.4 — UML Class Diagram

3.3 Methodology

Methodology refers to the comprehensive, organised set of practices used to carry out the project. The system was built using the Agile methodology, an iterative approach in which work is delivered in short, repeating cycles called sprints rather than in a single long phase. Each two-week sprint moved through planning, design, development, and testing, and concluded with a review and retrospective in which the team reflected on what went well and what could be improved. This cycle then repeated, allowing requirements to be refined continuously and working software to be produced incrementally. Automated testing with Selenium and Cucumber was embedded within each sprint, and Jenkins ran the test suite on every integration so that defects were caught early. The Agile workflow is shown in Figure 3.5.

Figure 3.5 — Agile Sprint Workflow



CHAPTER 4

SYSTEM IMPLEMENTATION

This chapter describes how the design was turned into a working system. The implementation was organised into clearly bounded modules, each developed and tested in turn across successive sprints. Development followed a test-first discipline wherever practical: behaviour was specified in Cucumber scenarios, the corresponding feature was implemented in Spring Boot and Angular, and the automated Selenium tests confirmed the behaviour before the work was considered complete.

Skill and Employee Management Module. This module provides full CRUD (Create, Read, Update, Delete) operations for employee records and their associated skills. Each employee is stored with a profile and a list of skills, and the manager can maintain this information through dedicated Angular screens backed by Spring Boot REST endpoints.

Task Management Module. Managers create tasks, each of which specifies a title, a description, and the skill it requires. Tasks carry a status field — for example, Open, In Progress, or Completed — that reflects their position in the workflow.

Allocation Engine. This is the core module. When a task needs to be assigned, the allocation service examines the skill the task requires and searches the employee records for those who possess a matching skill. It then ranks the suitable candidates and produces an allocation linking the task to the chosen employee. The allocation is persisted so that both manager and employee can see it.

Status Tracking Module. Once a task has been allocated, the assigned employee can update its status as work progresses. These updates are reflected back to the manager, providing visibility over the state of all work in the system.

Automated Test Suite. Running alongside the application code, the test suite uses Cucumber feature files to express scenarios in plain language and Selenium to drive the browser through those scenarios. The suite was executed automatically by Jenkins on each integration, ensuring that new changes did not break existing behaviour.

Table 4.1 — Core Modules Implemented

Module	Key Responsibility
Skill & Employee Management	Maintain employee profiles and their skill sets (CRUD)
Task Management	Create tasks with required skills and track their status
Allocation Engine	Match required skills to employees and assign tasks
Status Tracking	Allow employees to update progress on assigned tasks
Automated Test Suite	Validate behaviour end-to-end via Selenium and Cucumber

The modular structure meant that each part could be built, tested, and reviewed independently before being integrated, which kept the codebase stable throughout the internship and made it straightforward to demonstrate working increments at each sprint review.

CHAPTER 5

RESULTS AND DISCUSSIONS

The internship produced a complete, working Skill-Based Task Allocation System and, alongside it, a clear set of measurable outcomes against the organisation's evaluation framework. This chapter discusses both the technical results and the formal assessment results.

Functional Result. The delivered system successfully automates task allocation. A manager can record employees and their skills, create tasks with specific skill requirements, and have the system identify and assign the most suitable employee. Employees can view their assigned work and update its status, giving managers continuous visibility. The manual guesswork described in the problem statement was replaced by a consistent, rule-based process, which was the central goal of the project.

Quality Result. Because every feature was accompanied by automated Selenium and Cucumber tests running through Jenkins, the application remained stable as it grew. Regressions were caught quickly, and each sprint produced a demonstrable, tested increment of working software.

Evaluation Result. Cognizant assesses interns through a Qualifier assessment and a Red-Amber-Green (RAG) status applied at interim and final evaluations, where Green denotes a fully satisfactory outcome and is the required result. The candidate cleared the Qualifier with a score of 83 out of 100 against a mandatory pass mark of 70, achieved Green at the interim evaluation, and achieved Green at the final evaluation on the first attempt, with the final evaluation completed on 10 June 2026. These outcomes are summarised in Table 5.1.

Table 5.1 — Evaluation Outcomes

Evaluation	Result	Requirement
Qualifier Assessment	83 / 100	70 to pass
Interim Evaluation (RAG)	Green	Green required
Final Evaluation (RAG)	Green (first attempt)	Green required

Taken together, these results confirm that the internship met its objectives on both fronts: a useful application was delivered, and the formal benchmarks set by the organisation were comfortably satisfied

CHAPTER 6

CONCLUSION AND FUTURE WORK

Conclusion. The internship at Cognizant in the QEA-SDET domain achieved its aims. A complete Skill-Based Task Allocation System was designed, built, tested, and delivered using a professional full-stack technology suite — Java and Spring Boot, Angular, and SQL — with quality enforced throughout by Selenium, Cucumber, GitHub, and Jenkins, all within an Agile delivery process. The project solved a genuine problem by replacing manual, bias-prone task assignment with an automated, skill-driven process. Equally important, the internship built lasting professional competence: practical full-stack development skill, a test-first engineering mindset, fluency with industry tooling, and the discipline of working within an iterative team. The strong evaluation results — a Qualifier score of 83 out of 100 and Green status at both interim and final evaluations on the first attempt — reflect the quality of the work produced.

Future Work. Several enhancements could extend the system further. The matching logic could be upgraded from a rule-based approach to one that also weighs an employee’s current workload and past performance, producing fairer and more balanced allocations. A skill-proficiency level could be factored in so that more demanding tasks are routed to more experienced staff. Notifications could alert employees instantly when a task is assigned. A reporting and analytics dashboard could surface trends such as skill gaps within a team, helping managers plan training. Finally, the application could be containerised and deployed to the cloud with an extended CI/CD pipeline to support real production use at scale.

CHAPTER 7

APPENDICES

7.1 Source Code

This section presents representative source-code excerpts from the Skill-Based Task Allocation System. The excerpts illustrate the core entities, the allocation logic, a REST controller, and a Cucumber test scenario. They are shown for illustration and are not the complete codebase.

Listing 7.1 — Employee and Skill Entities (Java / Spring Boot)

```
@Entity
public class Employee {
    @Id @GeneratedValue
    private Long id;
    private String name;

    @ManyToMany(fetch = FetchType.EAGER)
    private List<Skill> skills = new ArrayList<>();

    // getters and setters
    public List<Skill> getSkills() { return skills; }
}

@Entity
public class Skill {
    @Id @GeneratedValue
    private Long id;
    private String name;
    private int level;    // 1 = basic, 5 = expert
}
```

Listing 7.2 — Allocation Service (Skill-Matching Logic)

```
@Service
public class AllocationService {

    @Autowired
    private EmployeeRepository employeeRepo;

    // Find employees whose skills match the task requirement
    public List<Employee> matchSkill(Skill required) {
        return employeeRepo.findAll().stream()
            .filter(e -> e.getSkills().stream()
                .anyMatch(s -> s.getName()
                    .equalsIgnoreCase(required.getName())))
            .collect(Collectors.toList());
    }

    // Allocate a task to the best-matched employee
    public Allocation allocateTask(Task task) {
        List<Employee> candidates = matchSkill(task.getReqSkill());
        if (candidates.isEmpty()) {
            throw new NoMatchException("No employee with the required skill");
        }
    }
}
```

```
    }  
    Employee chosen = candidates.get(0);  
    return new Allocation(task, chosen, LocalDate.now());  
  }  
}
```

Listing 7.3 — REST Controller (Spring Boot)

```
@RestController  
@RequestMapping("/api/tasks")  
public class TaskController {  
  
    @Autowired  
    private AllocationService allocationService;  
  
    @PostMapping("/{id}/allocate")  
    public ResponseEntity<Allocation> allocate(@PathVariable Long id) {  
        Task task = taskService.findById(id);  
        Allocation result = allocationService.allocateTask(task);  
        return ResponseEntity.ok(result);  
    }  
}
```

Listing 7.4 — Cucumber Feature Scenario (BDD)

```
Feature: Skill-based task allocation  
  
Scenario: Allocate a task to a matching employee  
    Given an employee "Avi" with the skill "Java"  
    And a task "Build REST API" that requires the skill "Java"  
    When the manager allocates the task  
    Then the task should be assigned to "Avi"  
    And the task status should be "In Progress"
```

7.2 Learning Experiences

Knowledge Acquired. The internship deepened and made practical the knowledge first encountered in academic courses. Concepts from object-oriented programming came alive in the Java and Spring Boot codebase; database theory was applied directly when designing the SQL schema and writing queries; and software-engineering principles such as separation of concerns and iterative development were experienced first-hand through the layered architecture and Agile sprints. Testing, often a theoretical topic in coursework, became a daily practice through Selenium and Cucumber.

Skills Learned. Beyond technical knowledge, the internship built genuine abilities: writing clean, maintainable full-stack code; designing and automating tests; using version control and continuous integration as part of normal work; reading and understanding requirements; breaking large problems into manageable modules; and collaborating effectively within a team. The ability to debug systematically and to seek and act on code-review feedback grew considerably.

Observed Attitudes and Gained Values. Working alongside experienced engineers highlighted the professional attitudes that underpin a successful career. Among the most important were discipline and consistency in writing tested code, dependability in honouring sprint commitments, openness in giving and receiving constructive feedback, and attention to detail in ensuring quality. The value of clear communication — keeping the team informed during stand-ups and reviews — was especially evident.

The Most Challenging Task. The most challenging task was designing and implementing the allocation engine so that it reliably matched tasks to employees while handling edge cases, such as a task for which no suitably skilled employee existed. The difficulty lay in getting the matching logic correct and in writing automated tests that proved it behaved correctly under all conditions. The challenge was overcome by breaking the problem into small, testable pieces, specifying the expected behaviour first in Cucumber scenarios, implementing the logic incrementally, and using the feedback from the automated test runs to refine the solution until it was robust.

7.3 SWOT Analysis

The following SWOT analysis reflects on the organisation and the internship experience, examining internal Strengths and Weaknesses alongside external Opportunities and Threats.

Strengths. The organisation provides a highly structured, mentor-supported internship within a globally recognised technology company. Access to industry-standard tools, established engineering processes, and experienced senior engineers creates an excellent learning environment. The emphasis on automated quality engineering and Agile delivery equips interns with directly employable, modern skills.

Weaknesses. The breadth of the technology stack — spanning back end, front end, database, and a full testing pipeline — can be demanding for a newcomer within a limited time frame, and the depth that can be reached in any single tool is naturally constrained by the internship’s duration. The structured environment, while supportive, leaves limited room for exploring alternative approaches outside the prescribed process.

Opportunities. The skills gained — full-stack development with Java, Spring Boot, and Angular, combined with SDET-style test automation — are in strong demand across the industry, opening clear pathways to roles in software engineering and quality engineering. The exposure to a large enterprise’s working practices provides a strong foundation for a professional career and for future specialisation in areas such as cloud or DevOps.

Threats. The IT services sector is highly competitive and evolves rapidly; tools and frameworks change quickly, so skills must be continually refreshed to stay relevant. Increasing automation and the rise of AI-assisted development mean that engineers must keep adding higher-level capabilities to remain valuable. These external pressures make lifelong learning essential.